SkillSwap – Project Documentation

Oct 13, 2026 · @Vrun

1. Introduction

SkillSwap is a web application where college students teach a skill they already know in exchange for learning a skill they want, so every connection is a two-way swap rather than a one-way favour.

Most student platforms only match people with similar interests. SkillSwap matches reciprocal needs: if Student A knows Java and wants to learn Photoshop, and Student B knows Photoshop and wants to learn Java, A can find B, view the profile and send an exchange request that B accepts or rejects.

Objectives

- Let students list the skills they can teach and the skills they want to learn.
- Help students discover suitable partners through skill search and match highlighting.
- Manage the full request cycle: send, receive, accept, reject and track status.
- Give an administrator control over users, skills and exchange requests.
- Keep the code simple enough to explain line by line in a practical exam.

Technology stack

Layer	Technology
Frontend	HTML, CSS, a little inline JavaScript for delete confirmations
Backend	Core PHP 8 with MySQLi prepared statements
Database	MySQL (InnoDB, foreign keys)
Authentication	PHP sessions, `password_hash` / `password_verify`
Development setup	PHP built-in server (`php -S`) with a local MySQL; Docker as backup

2. Users and roles

SkillSwap has two roles, stored in the `role` column of the `users` table: students do the swapping, and an administrator keeps the platform clean.

Role	How the account is created	Can do	Cannot do
Student	Self-registration on `register.php`	Edit own profile and skills, search students, view profiles, send requests, accept or reject requests received, track sent requests	Open admin pages, change another student's profile or skills, send a request to themselves, answer a request addressed to someone else
Administrator	Inserted by `database.sql` (`admin@skillswap.com`)	View platform statistics, list users and delete students, add and delete skills, view every exchange request with its status	Delete their own account, use the student pages (they are redirected to the admin panel)

The administrator's role is read from the database again on every admin page and action, so changing a URL or editing the session cannot grant admin access.

3. Functionality

The student side covers nine features, from registration to accepting a swap, and the admin side is a four-tab panel. A **perfect match** is a student who teaches something you want to learn and wants something you can teach.

Student features

Feature	Page	What it does
Register	`register.php`	Validates input, rejects a duplicate email, stores a hashed password, then redirects to login
Login and logout	`login.php`, `logout.php`	Starts or destroys the session; one generic error message for wrong credentials
Dashboard	`dashboard.php`	Live counts of skills offered, skills wanted, pending requests received and accepted exchanges, plus up to three perfect matches
Profile	`profile.php`	Edit name, course, year and bio
My skills	`skills.php`	Add or remove skills the student can teach (OFFER) or wants to learn (WANT), chosen from the skills table
Find students	`students.php`	Search by skill name; cards show offered and wanted skills and a match badge; the logged-in student never appears
Student profile	`student.php`	Public profile with skills that match the viewer highlighted, and the swap request form
Send request	`request.php`	Checks both skills, blocks requests to self and duplicate pending requests, saves the request as PENDING
My requests	`requests.php`	Received and sent requests as cards; the receiver can accept or reject a pending request

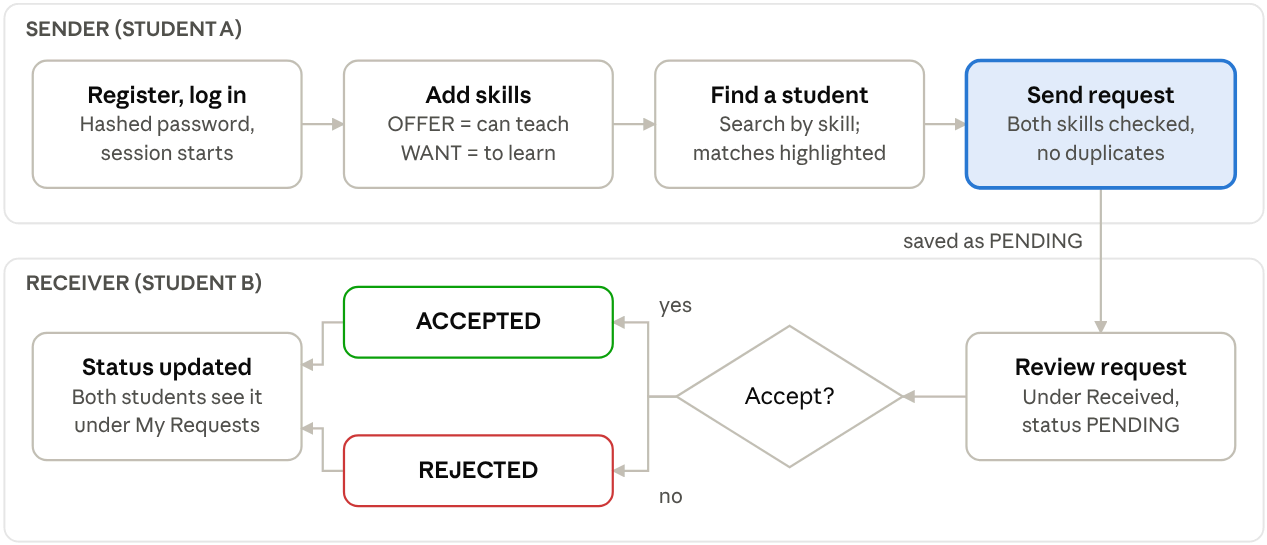
Administrator features (all on `admin.php`)

Tab	What it does
Overview	Real counts of students, skills, exchange requests and pending requests
Users	List all users; delete a student (their skills and requests are removed with them)

Tab	What it does
Skills	Add a new skill with a category; delete a skill
Requests	View every exchange request with sender, receiver, skills, date and status

4. Application workflow

A swap runs from registration to a decision in seven steps, and only the receiver can decide.



swap workflow · two lanes, one decision

The sender's request is saved as PENDING; the receiver's accept or reject then updates the status that both students see.

1. **Register and log in.** The student creates an account and logs in; the session stores their user ID and role.
2. **Add skills.** In My Skills the student lists what they can teach (OFFER) and what they want to learn (WANT).
3. **Find a student.** Search by skill name; a card marked perfect match teaches what you want and wants what you teach.
4. **Open the profile.** Matching skills are highlighted, and the swap form lists your OFFER skills and the student's OFFER skills.

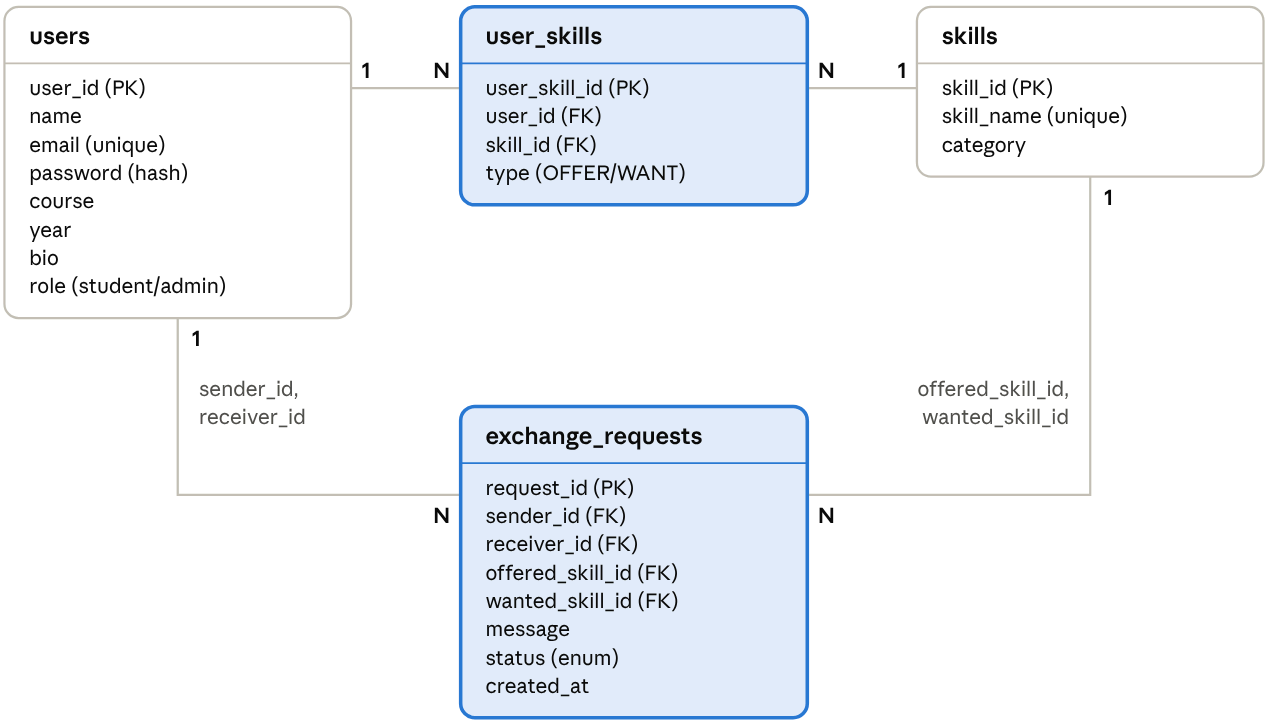
5. **Send the request.** `request.php` validates both skills and saves the request as PENDING.
6. **Receiver decides.** Under My Requests the receiver accepts or rejects; only the receiver of a pending request can do this.
7. **Both see the result.** The Sent and Received lists show ACCEPTED or REJECTED.

5. Database design

The `skillswap` database has four InnoDB tables: `users` and `skills` hold the entities, `user_skills` links a student to a skill as OFFER or WANT, and `exchange_requests` records each swap proposal.

Students and skills connect through two link tables

Highlighted tables are link tables. PK = primary key, FK = foreign key. 1 and N show how many rows can be linked.



ER diagram · 4 tables, 4 relationships

A student has many `user_skills` rows and a skill appears in many of them; every exchange request points to two students and two skills.

users

Column	Type	Key or rule
`user_id`	INT, auto-increment	Primary key
`name`	VARCHAR(100)	Required
`email`	VARCHAR(150)	Required, unique
`password`	VARCHAR(255)	Required, bcrypt hash
`course`	VARCHAR(100)	Optional
`year`	VARCHAR(30)	Optional
`bio`	TEXT	Optional
`role`	ENUM('student','admin')	Default `student`

skills

Column	Type	Key or rule
`skill_id`	INT, auto-increment	Primary key
`skill_name`	VARCHAR(100)	Required, unique
`category`	VARCHAR(50)	Required (Technology, Creative, Academic, Lifestyle)

user_skills

Column	Type	Key or rule
`user_skill_id`	INT, auto-increment	Primary key
`user_id`	INT	Foreign key to `users`, ON DELETE CASCADE
`skill_id`	INT	Foreign key to `skills`, ON DELETE CASCADE
`type`	ENUM('OFFER','WANT')	Required; (`user_id`, `skill_id`, `type`) is unique

exchange_requests

Column	Type	Key or rule
`request_id`	INT, auto-increment	Primary key
`sender_id`	INT	Foreign key to `users`, ON DELETE CASCADE
`receiver_id`	INT	Foreign key to `users`, ON DELETE CASCADE
`offered_skill_id`	INT	Foreign key to `skills`, ON DELETE CASCADE
`wanted_skill_id`	INT	Foreign key to `skills`, ON DELETE CASCADE
`message`	TEXT	Optional
`status`	ENUM('PENDING','ACCEPTED','REJECTED','COMPLETED')	Default `PENDING`
`created_at`	TIMESTAMP	Default current time

6. Project structure

The project is a flat set of core PHP pages plus an `includes/` folder for code every page shares.

File	Purpose
`index.php`	Public landing page with a live student count

File	Purpose
`register.php`, `login.php`, `logout.php`	Account creation, authentication, session end
`dashboard.php`	Student home with live statistics and best matches
`profile.php`	View and edit the student's own profile
`skills.php`	Manage OFFER and WANT skills
`students.php`, `student.php`	Student search and a single public profile
`request.php`	Processes the swap request form
`requests.php`	Sent and received requests, accept and reject
`admin.php`	Administrator panel (overview, users, skills, requests)
`db.php`	MySQL connection settings
`database.sql`	Creates the database, four tables and sample data
`includes/helpers.php`	Session start, `e()` escaping, flash messages, CSRF, login and admin checks
`includes/header.php`, `footer.php`	Shared navbar and footer for every page
`includes/cards.php`	Student cards, skill chips, match query, SVG icons
`styles.css`	Single stylesheet for all pages

7. Security and validation

Every check runs on the server, so it still applies if someone edits the page in the browser.

- **SQL injection:** every query that uses user input is a prepared statement.
- **Passwords:** stored with `password_hash`, checked with `password_verify`; the session ID is regenerated at login, and a wrong email or password gives the same message.
- **CSRF:** each form carries a session token that `csrf_check()` verifies before any change.

- **XSS:** all user text is escaped with `htmlspecialchars` through the `e()` helper.
- **Safe methods:** every change uses POST and ends with a redirect and a flash message.
- **Ownership:** a skill is removed only with `AND user_id = <session user>`; a request is accepted or rejected only with `receiver_id = <session user>` and status PENDING; the sender of a request always comes from the session, never the form.
- **Roles:** `require_admin()` re-reads the role from the database; students cannot reach admin pages and admins are redirected away from student pages.
- **Data integrity:** unique keys on email, skill name and (user, skill, type); foreign keys with `ON DELETE CASCADE` so deleting a user or skill leaves no orphan rows.

Form validation

Form	Checks
Register	Name and course required, valid email, password of 6 or more characters that matches the confirmation, year from a fixed list, email not already registered
Profile	Same field rules, bio limited to 500 characters
Add skill	Type is OFFER or WANT, the skill exists, not already added
Swap request	Not to self; receiver exists; offered skill is in the sender's OFFER list; wanted skill is in the receiver's OFFER list; no identical pending request; message at most 300 characters

8. Screenshots

Paste one screenshot under each caption below, logged in as the role shown.

Figure 1 – Home page (logged out)

Paste screenshot here

Figure 2 – Registration form (with a validation error shown)

Paste screenshot here

Figure 3 – Login page

Paste screenshot here

Figure 4 – Student dashboard (statistics and best matches)

Paste screenshot here

Figure 5 – My Skills (teach and learn columns)

Paste screenshot here

Figure 6 – Find students (search results with a perfect-match badge)

Paste screenshot here

Figure 7 – Student profile and swap request form (matching skills highlighted)

Paste screenshot here

Figure 8 – My Requests (received with Accept and Reject, and sent)

Paste screenshot here

Figure 9 – Admin overview and users tab

Paste screenshot here

Figure 10 – Admin skills and requests tabs

Paste screenshot here

9. Setup, test accounts and future scope

The project needs PHP 7.4 or newer with the `mysqli` and `mbstring` extensions, and a running MySQL or MariaDB server.

1. Put all project files in one folder.
2. Open `db.php` and set the MySQL user, password and port.
3. Run `database.sql` once (`mysql -u root -p < database.sql`). It creates the `skillswap` database, four tables, 16 skills and the test accounts. Running it again deletes all data.
4. In the project folder run `php -S localhost:8080`.
5. Open `http://localhost:8080` in a browser.

Test accounts

Role	Email	Password
Administrator	`admin@skillswap.com`	`admin123`
Student	`ruchit@test.com`, `rahul@test.com`, `meera@test.com`, `diya@test.com`	`student123`

Limitations and future scope

- The COMPLETED status exists in the database but has no button yet.
- No ratings or reviews, chat, email notifications or password reset.
- Search results are not paginated and form errors appear only after submitting.
- Possible next steps: mark an accepted swap as completed, rate a partner, and notify students by email.